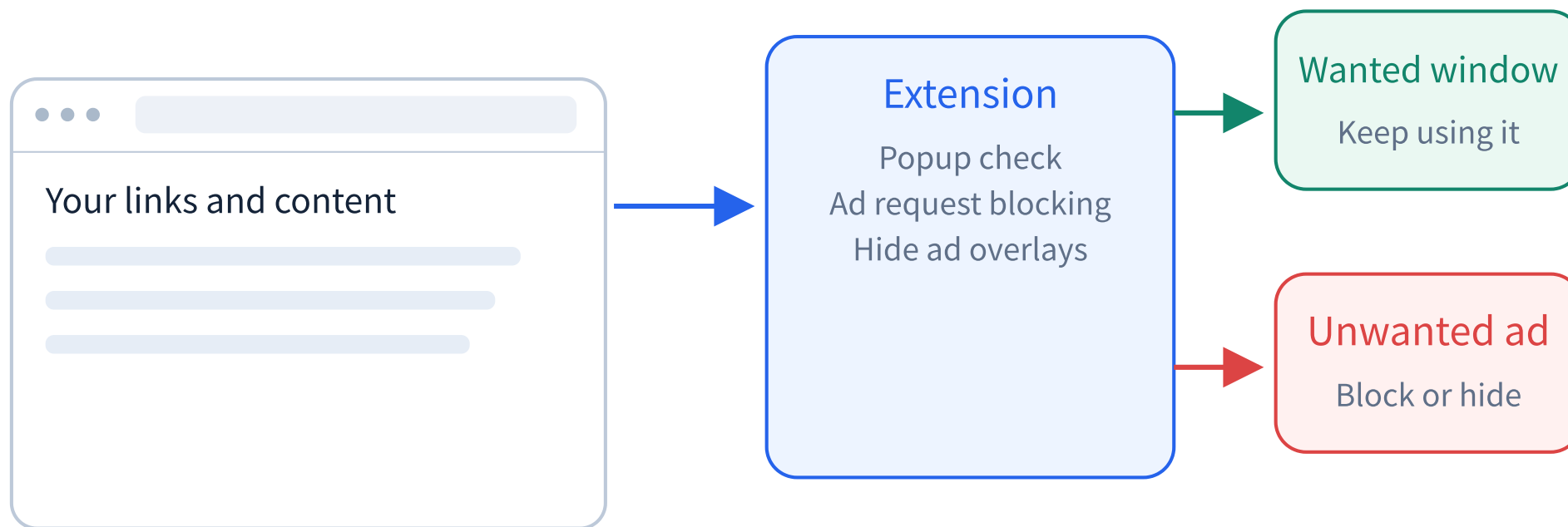


Keep wanted windows. Block unwanted ads.

Popup & Ad Overlay Blocker v1.3.0 | An illustrated guide for everyone



The goal is to preserve the windows you choose to open.

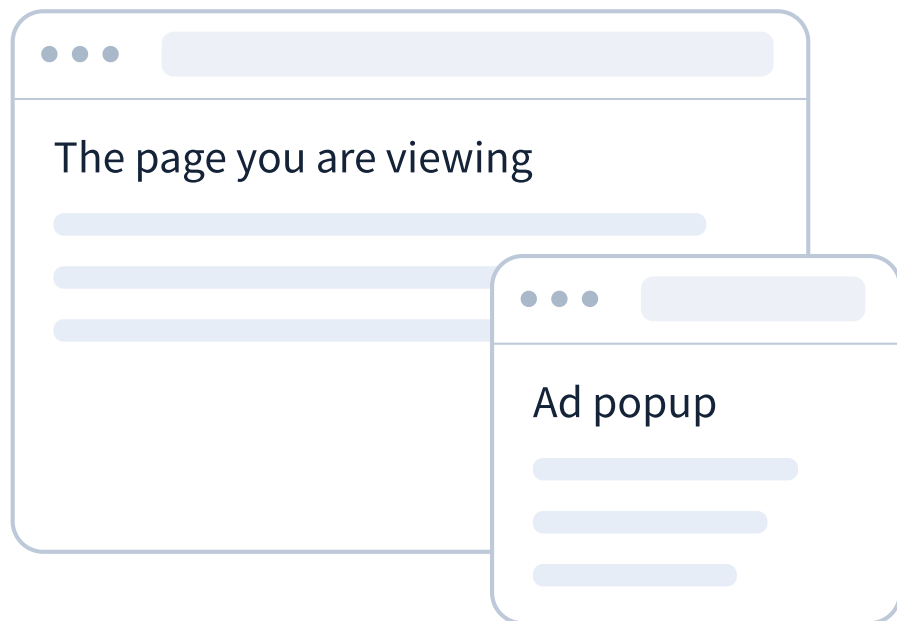
A Chrome extension that reduces ads interrupting your browsing.

Diagrams are illustrative. The extension cannot identify every ad perfectly.

Ads can interrupt browsing in two different ways

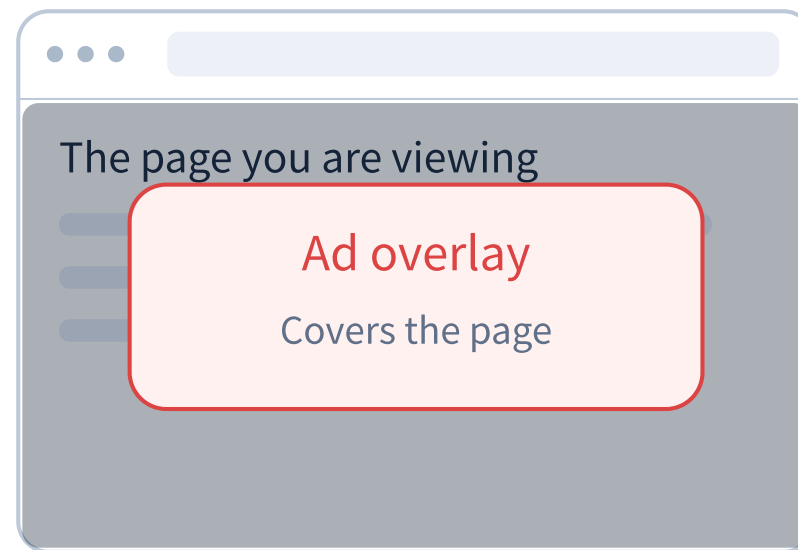
They look different and need different blocking methods.

Popup ad



A separate window or tab opens

Ad overlay

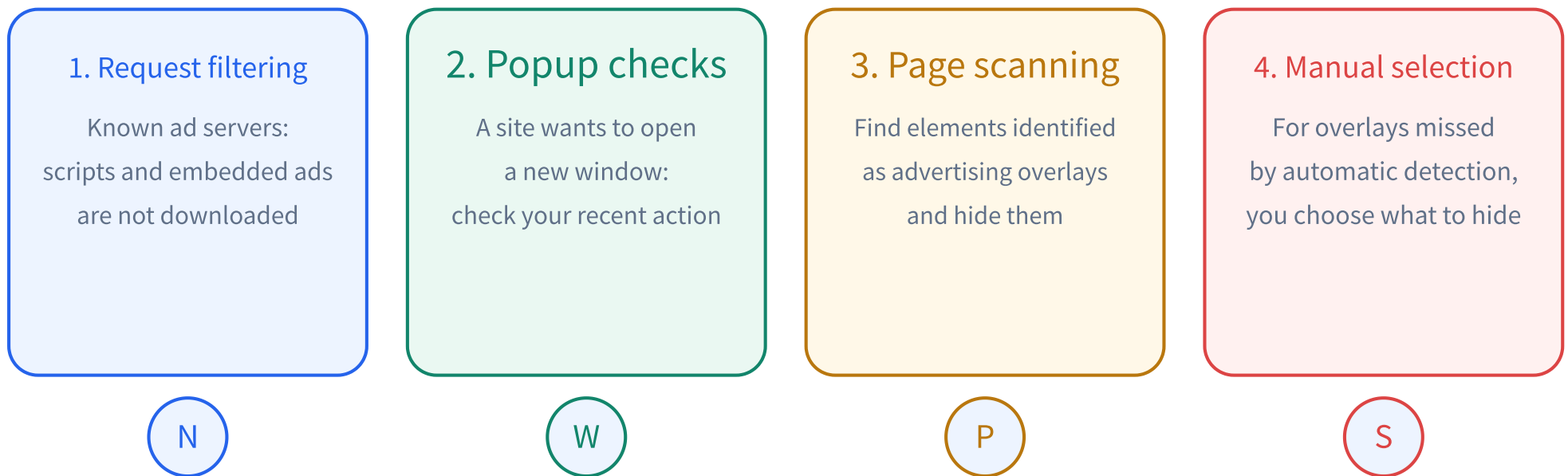


An extra layer covers the page

Popup checks handle new windows. Overlay hiding handles layers covering the page.

Four complementary roles

The roles complement one another; they do not always run in this order.

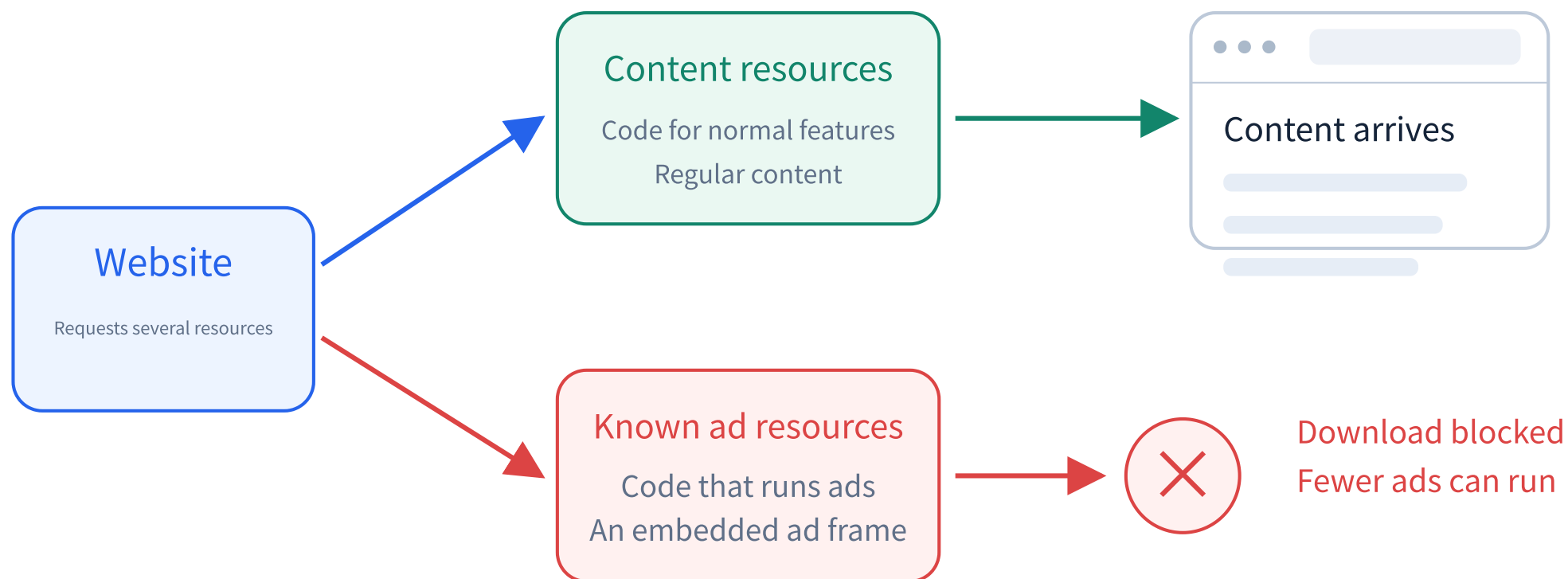


Three automatic checks + your own selections

Chrome's built-in popup blocking is also enabled as an additional safeguard.

1. Block known ads before they arrive

Chrome checks the server address and the type of resource requested.

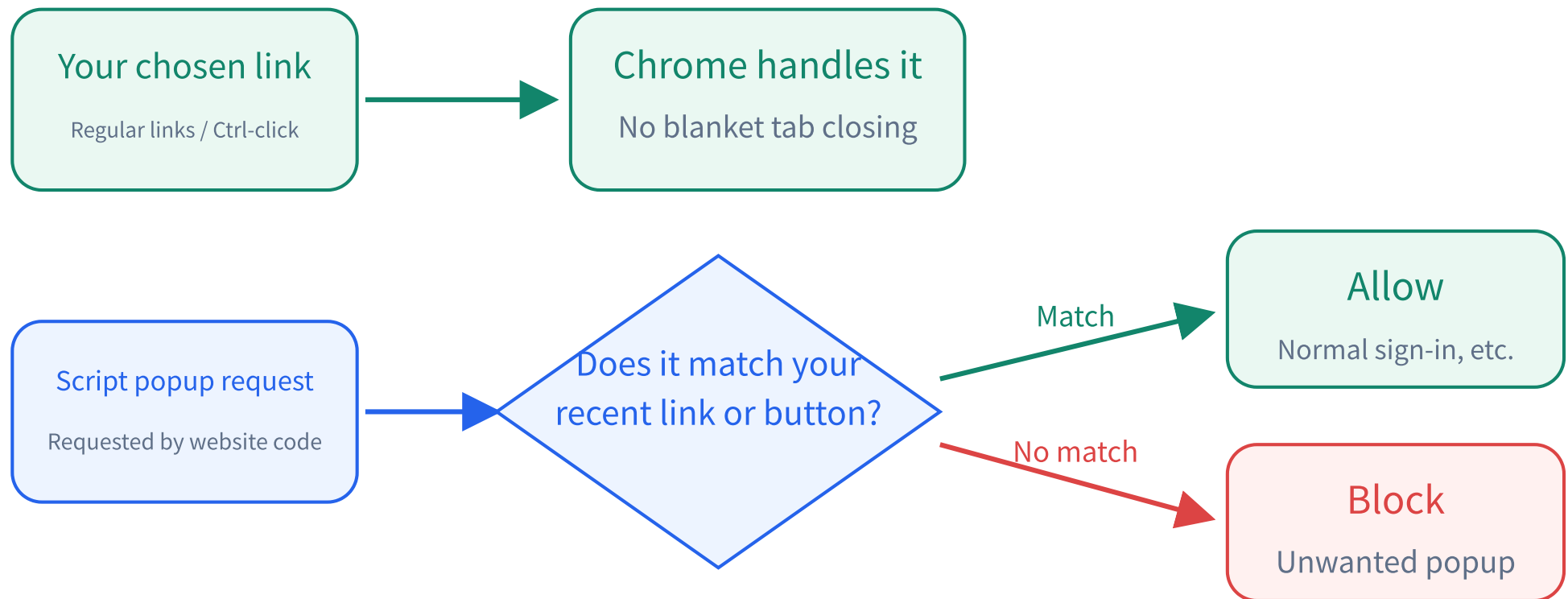


Regular page navigation is not blocked by these rules.

Rules target scripts and ad frames from listed servers. They do not remove every ad image.

2. Compare popup requests with your recent action

The key clue is whether you just selected a real link or button.



For links, the destination must match. Buttons normally allow popups, except to known ad hosts.

The time window is under one second, and Chrome must also recognize an active user gesture.

How are common situations handled?

The extension infers intent from your actions; it cannot read your mind.

1

Open a link in a new tab

The address you selected
Usually allowed

2

Click a sign-in button

A normal sign-in popup
Usually allowed

3

An ad opens after an empty-area click

No link or button selected
A blocking candidate

4

An ad is attached to a button

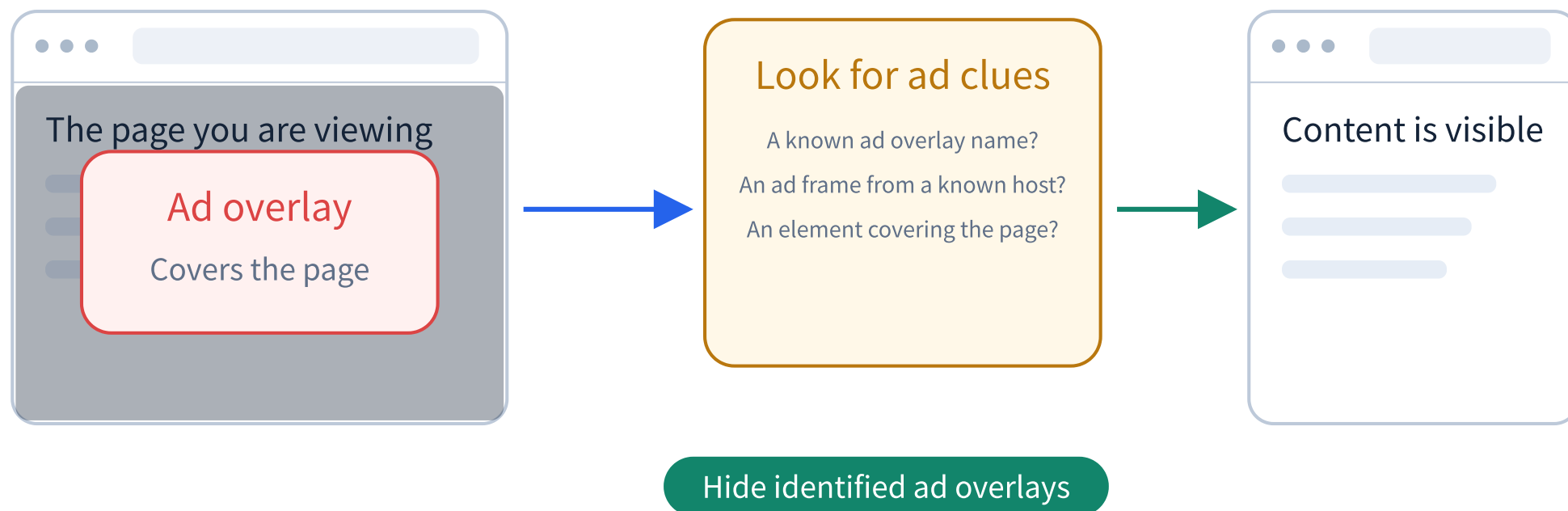
Hard to separate from normal use
Some ads may still get through

It cannot guarantee every wanted popup or perfectly identify every ad.

If a site's custom button is blocked, temporarily turn off blocking to use it.

3. Hide identified advertising overlays

An overlay is like placing another sheet on top of the page.

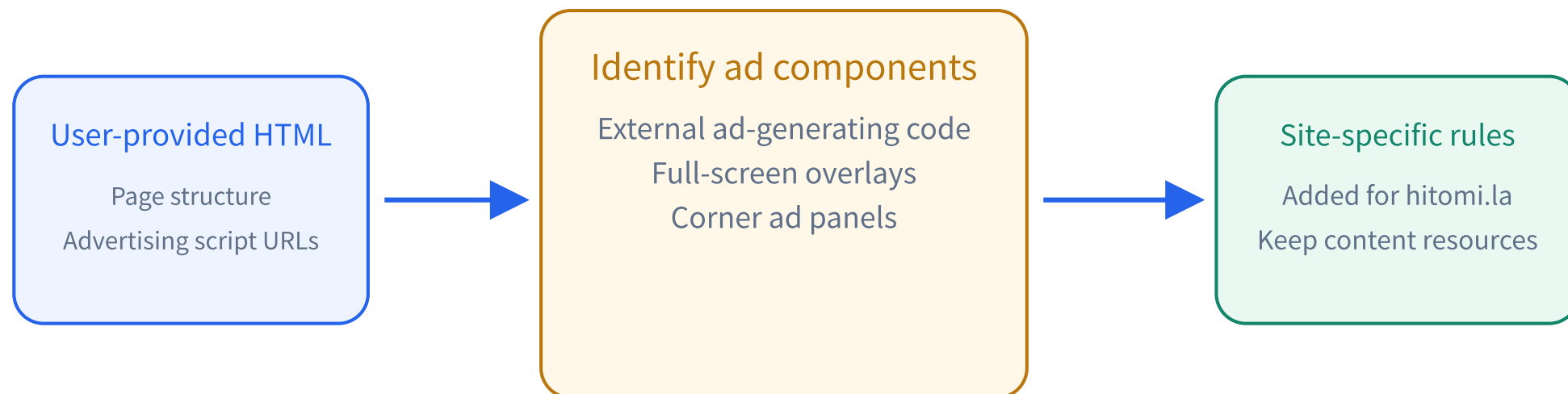


Look for advertising clues first, to avoid hiding ordinary menus or sign-in dialogs.

General automatic rules target large overlays. Confirmed site-specific rules also handle smaller ads.

Add targeted rules for the reported site

The supplied HTML identified additional ad hosts and overlay selectors.



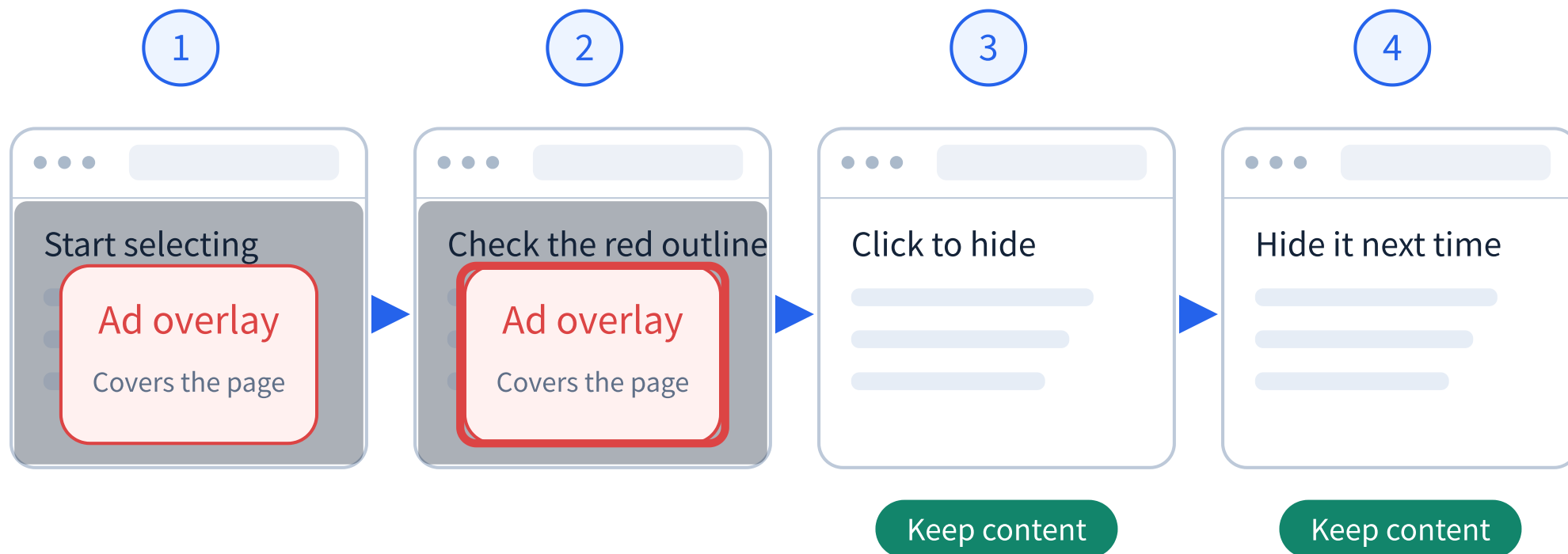
Varying overlay names can evade general rules.
Confirmed script hosts and overlay identifiers become site-specific rules.

Additional host rules are scoped to that site. General ad host rules apply across sites.

Live access was restricted, so testing used a page reproducing the supplied ad structures.

4. Select overlays that automatic checks miss

Extension menu → Select an ad overlay → Click the unwanted overlay



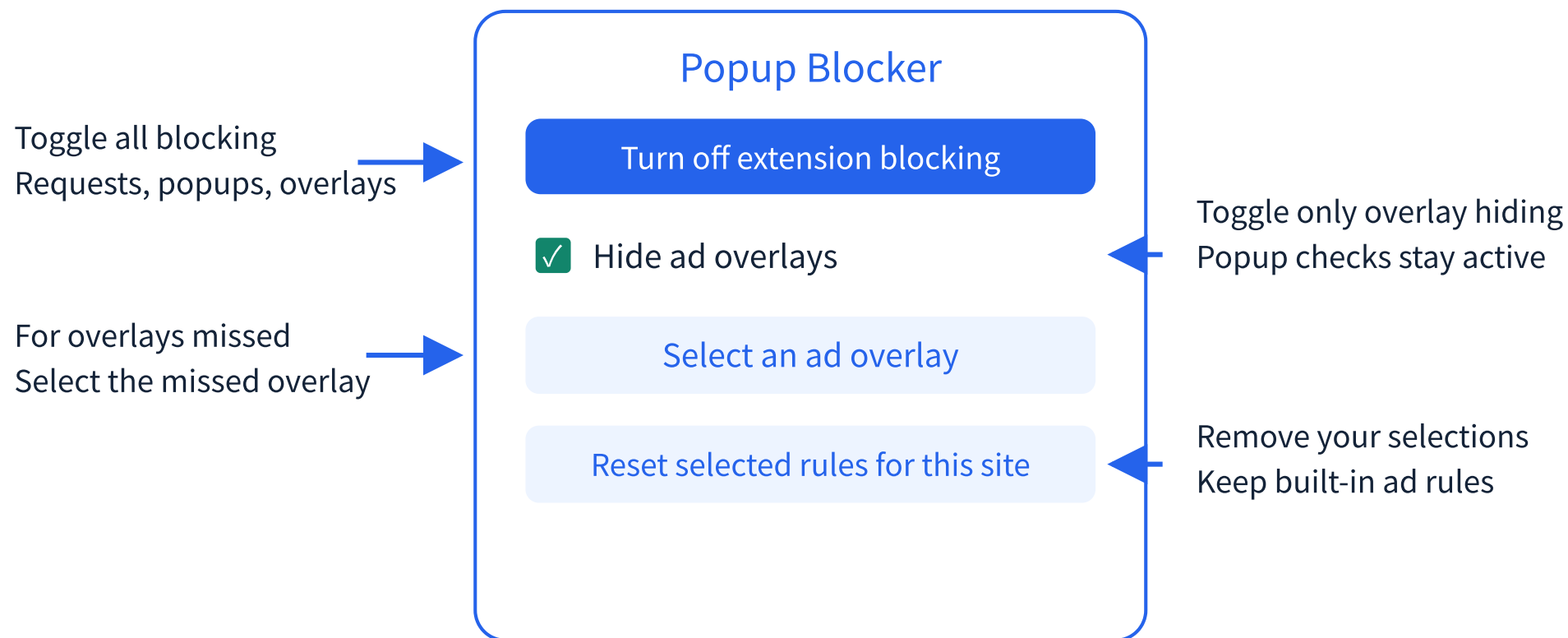
Select once: remember matching overlays on the same site

If the structure changes, select again. Esc cancels selection; reset site rules to undo a mistake.

Manual selection targets positioned overlays, not the whole page or ordinary page content.

Four menu controls, four different roles

Overall blocking and overlay hiding are separate settings.

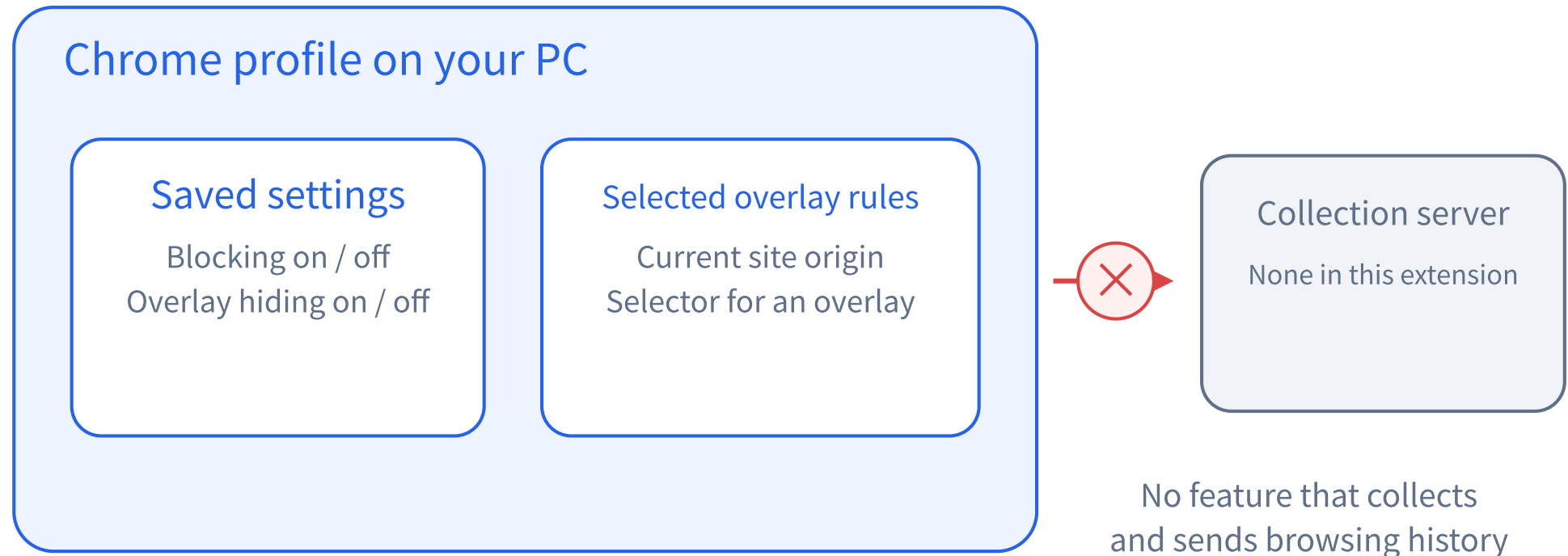


Turning off the extension restores Chrome's prior popup setting, which may still block popups.

This is an illustrative menu. Its labels are translated; the extension interface remains Korean.

Your rules are stored in your Chrome profile

Settings and selected rules are not automatically shared with other people's computers.



Removing and reinstalling clears your selections. Changed site structure can invalidate them.

This describes the extension itself. It does not stop all traffic or data collection by websites.

A useful helper, not a perfect ad detector

The aim is to reduce unwanted ads while preserving intentional windows.

Verified in tests

Block automatic and unrelated-click popups
Keep chosen links, Ctrl-click and sign-in
Hide overlays and handle recurrence

Remaining limitations

New ad hosts or changed overlays may be missed
Ads attached to buttons are hard to distinguish
Sites can bypass the page guard

If something still goes wrong

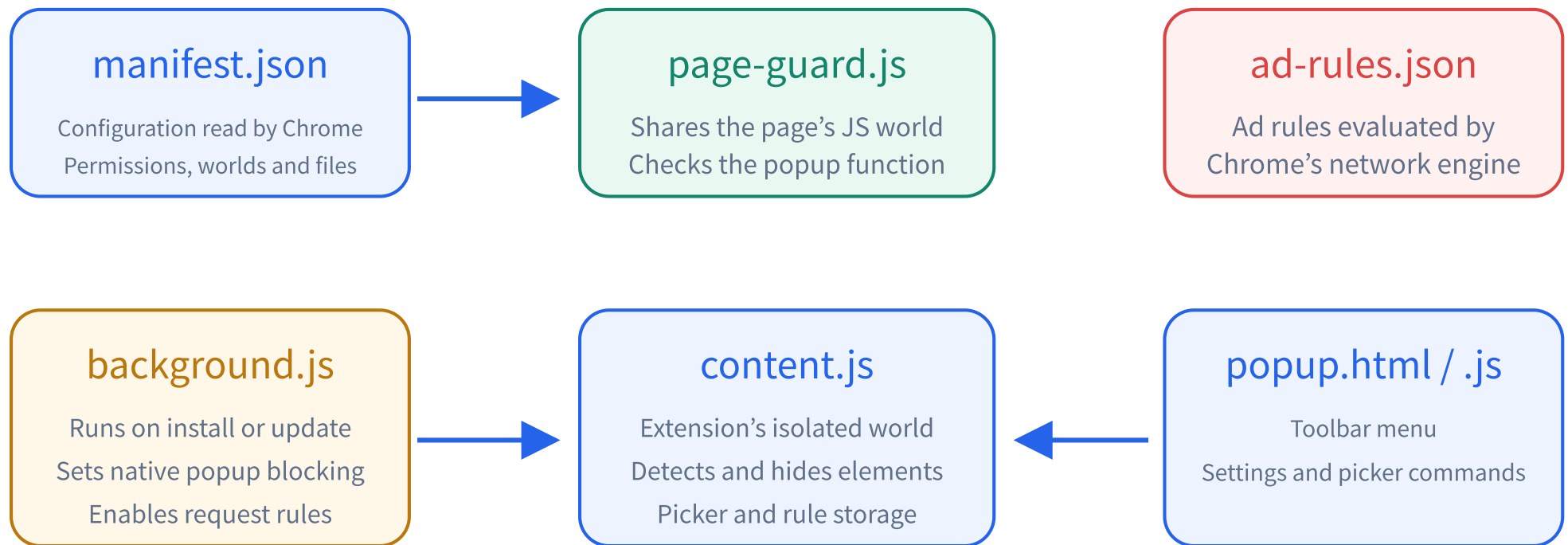
1. Refresh after updates.
2. Select remaining ad overlays.
3. Turn blocking off if needed.
4. Reset mistaken selections.

It does not certify site safety or block windows opened by external programs.

Unit and Chromium fixture tests passed. Extension installation, network-rule integration and live-site behavior remain unverified due to policy restrictions.

Technical 1: Files and execution worlds

JavaScript (.js) runs behavior; JSON (.json) contains configuration and rules.



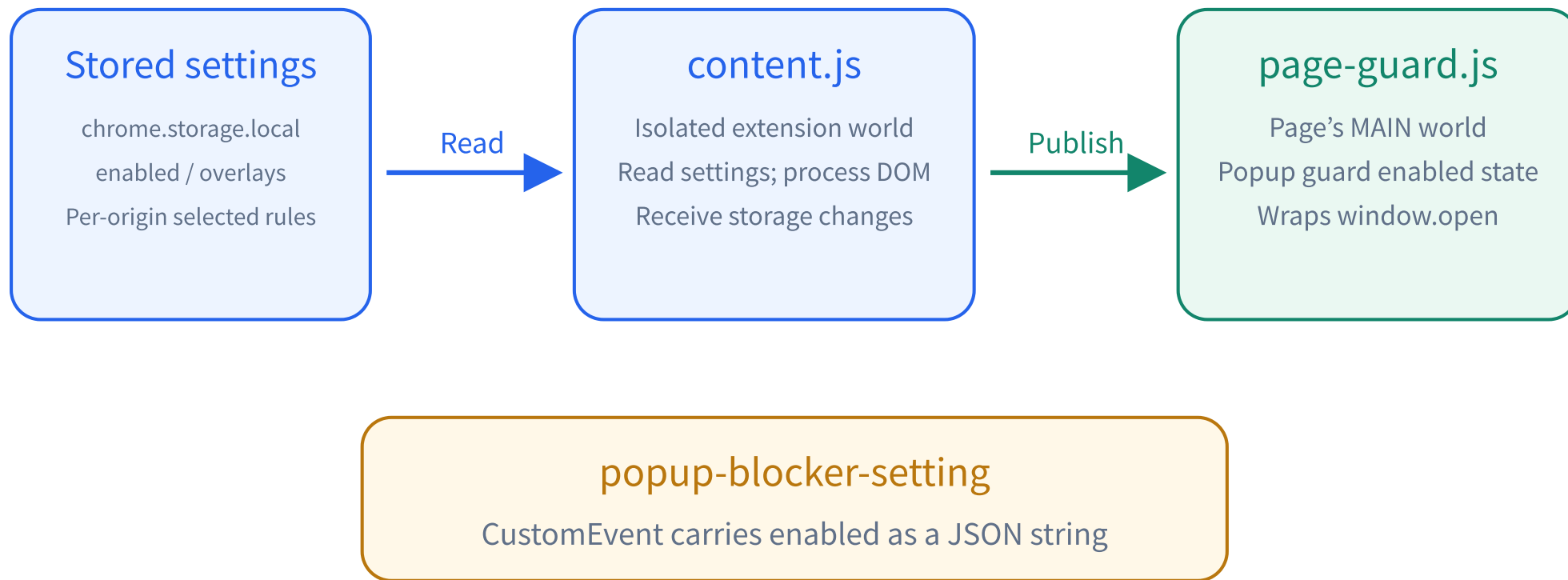
Separate execution worlds exchange only necessary settings.

The background is an event-driven service worker started by Chrome, not an always-running server.

The manifest uses Manifest V3. Content scripts start at `document_start` and also run in matching frames.

Technical 2: Delivering settings to the page

Both worlds share the document, but their JavaScript variables are separate.



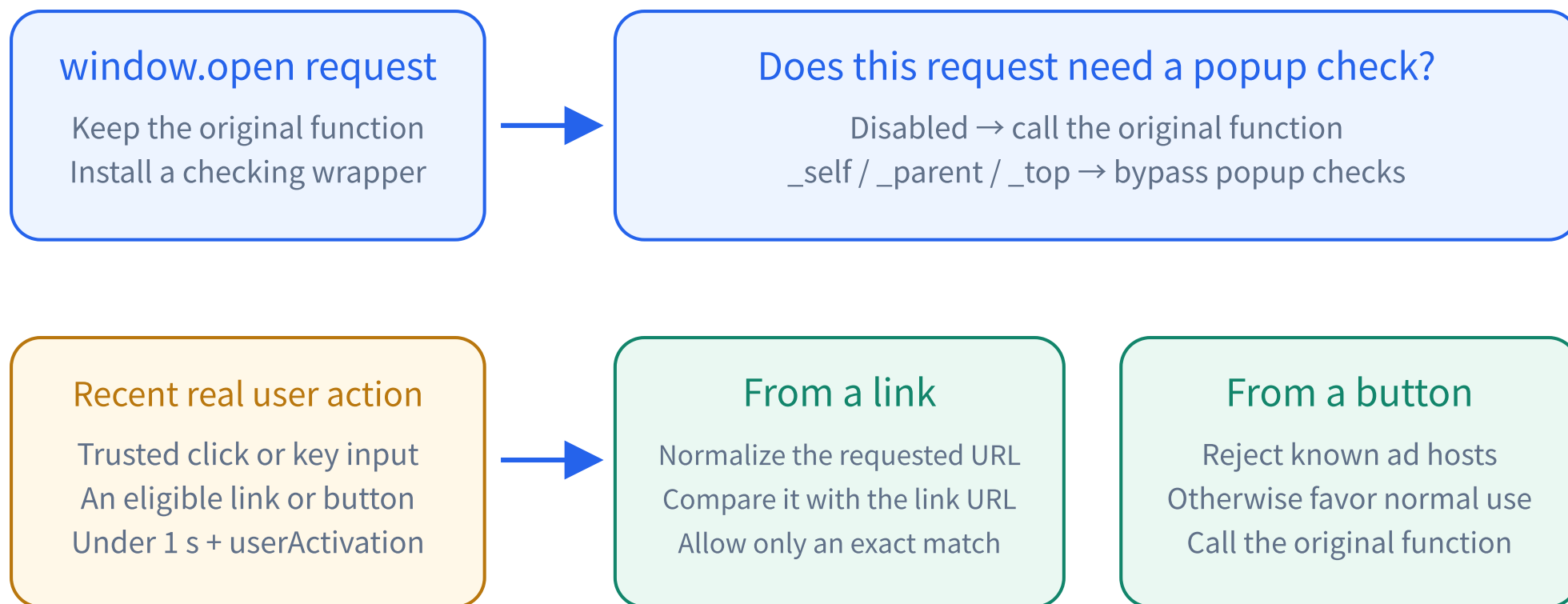
Storage changes → refresh settings → update page behavior

storage.onChanged triggers a refresh. content.js passes only enabled to the page guard.

Page scripts can tamper with this document event. It is a convenience bridge, not a security boundary.

Technical 3: Popup checks in the code

`window.open` is a browser function that page code uses to open a window or tab.

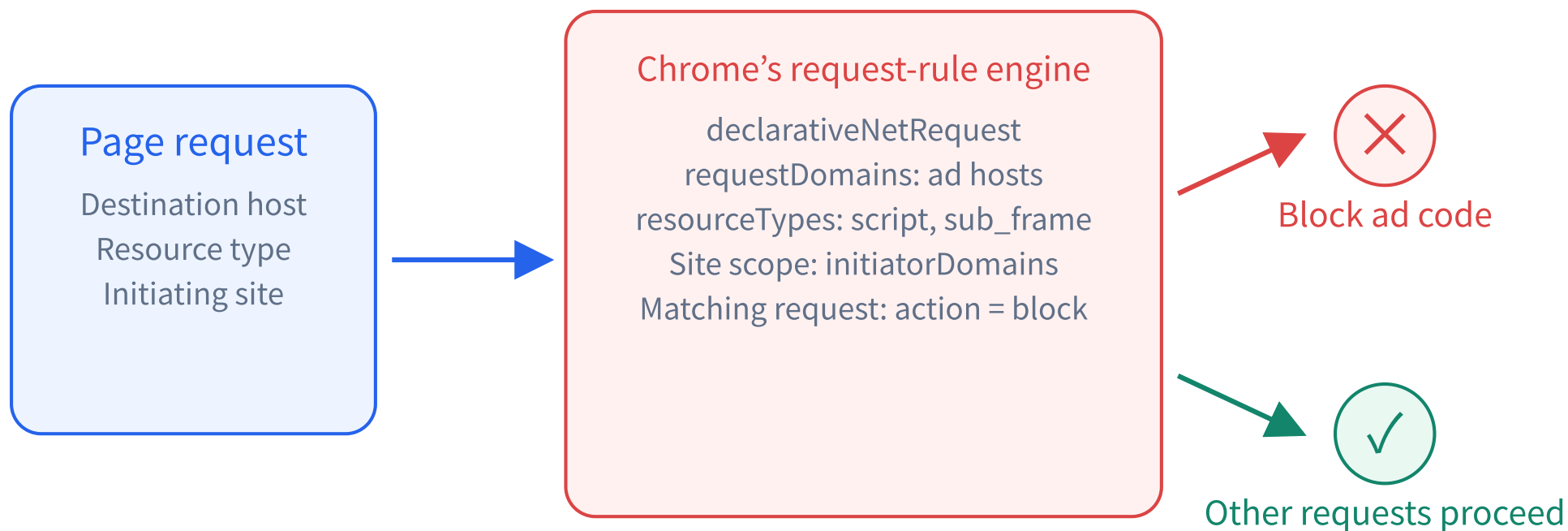


Allowed requests call the original via `Reflect.apply`. Blocked requests return null without opening a window.

Synthetic clicks are also checked with `preventDefault()`. This function uses seven basic ad hosts, separate from site-specific network rules.

Technical 4: Block ad code before execution

A script is external executable code. A sub_frame is a separate embedded web page.



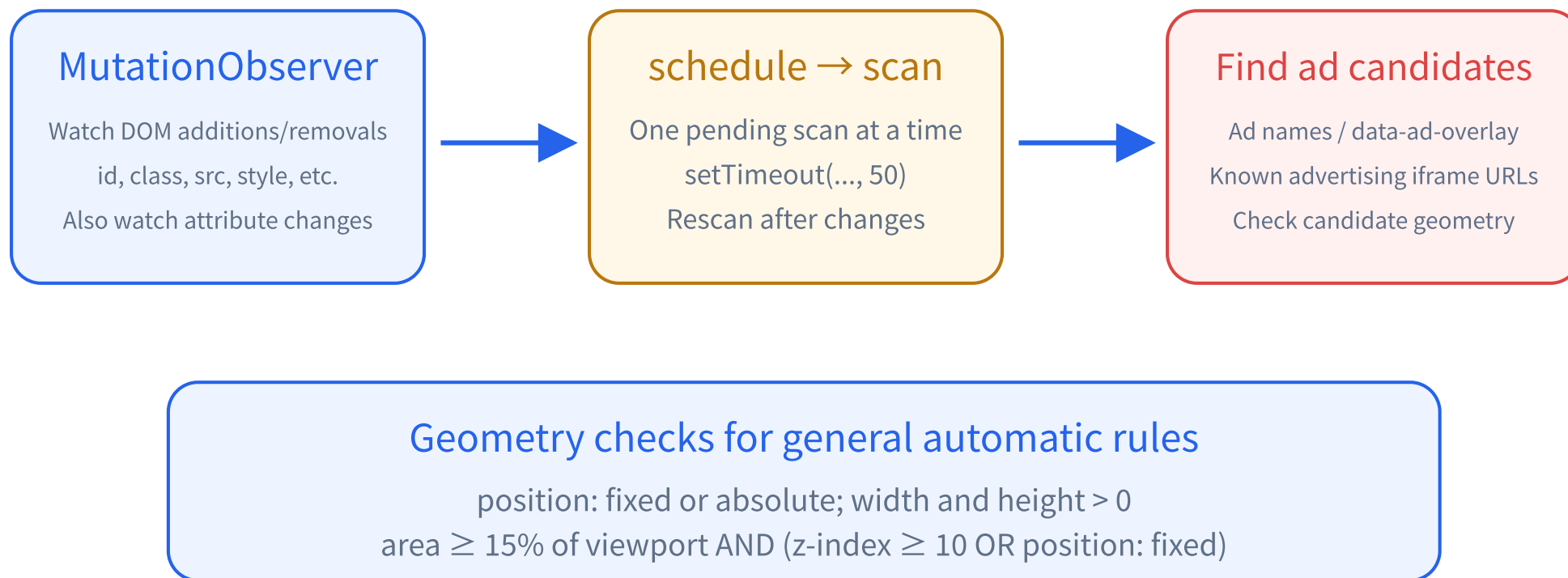
Current rules: general ad hosts + site-scoped hitomi.la hosts

Chrome applies these rules independently of the page wrapper. They do not target main_frame navigation.

popup.js uses updateEnabledRulesets to toggle the ruleset. Other resource types from those hosts are not blocked by these rules alone.

Technical 5: Detecting ad overlays

The DOM describes page elements. CSS controls their position, dimensions and appearance.



50 ms schedules a scan; it is not a strict execution deadline or an unconditional polling interval.

Confirmed site ad classes bypass size checks. Manually selected overlays omit the 15% threshold but still require positioned geometry.

Technical 6: Hiding and restoration

The Map is a temporary record pairing an element with its original inline display style.



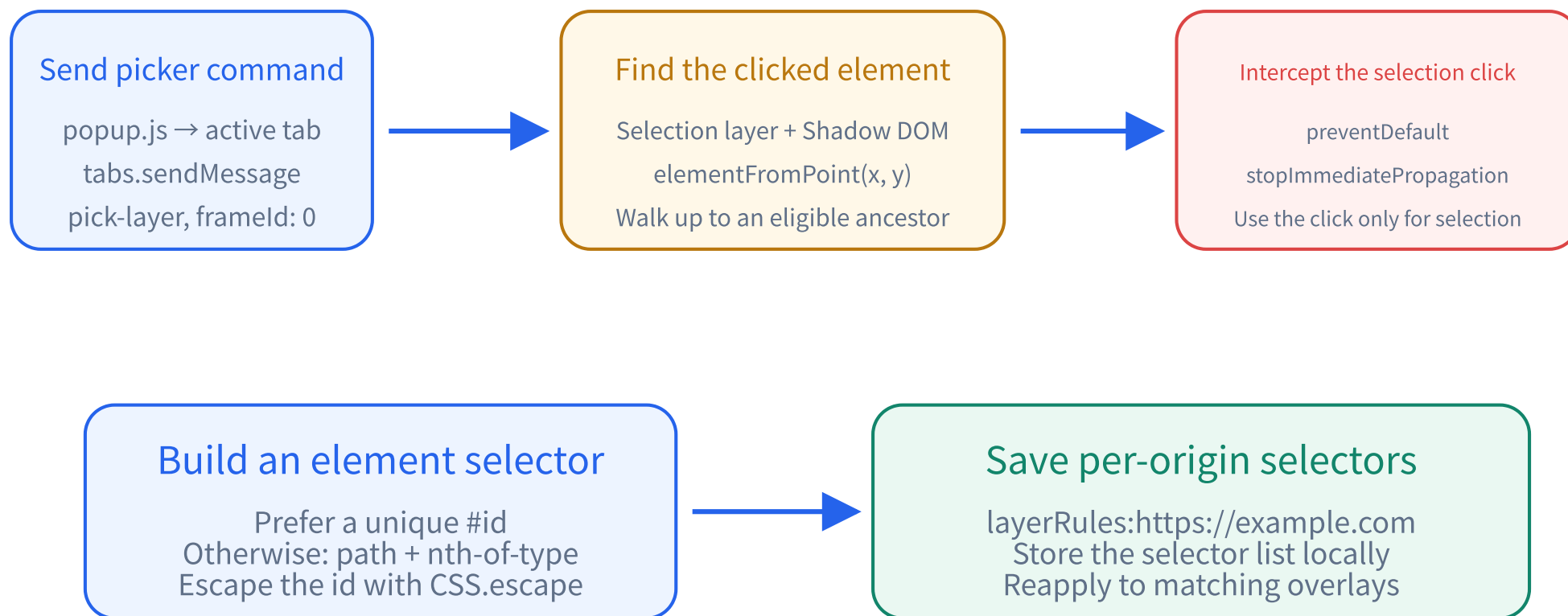
Hiding records live in page memory; selected rules live in Chrome storage.

Disabling blocking or overlay hiding restores styles. Changing or resetting selected rules restores elements before rescanning.

A page refresh creates new elements and a new Map. Saved selectors are read again and reapplied.

Technical 7: Remember an element's selector

A CSS selector is an expression used to find matching elements in the document.



Temporarily bypass the selection layer to find the element below. Finish or press Esc to remove the picker and its listeners.

Rules are scoped to an origin (scheme + host + port). Matching structures on other pages also apply; changed structures need a new selection.

Technical 8: Permissions, limits and validation

This appendix describes the current code. It does not introduce new extension functionality.

Required permissions

contentSettings: popup settings
storage: settings and site rules
declarativeNetRequest: block requests
activeTab: access the selected tab

Implementation limitations

Page code can bypass functions or events
Async settings read; initially enabled
Custom legitimate controls may be blocked
Changed selectors can miss an overlay

Separate behavior tests from integration tests

Verified: unit tests and shipped scripts running on Chromium fixture pages
Unverified: installed extension, network-rule integration and live target-site behavior

Network blocking uses Chrome's engine; popup and overlay behavior uses page scripts. Testing one does not validate the other.

Cloud policy blocks unpacked extensions and the proxy denies the target site. The extension is not a security boundary or comprehensive ad defense.